

Pattern Matching with Exception Handling

Document #: P2381R0

Date: 2021-04-25

Project: Programming Language C++

Audience: Evolution

Reply-to: Jarrad J. Waterloo
<descender76@gmail.com>

Contents

[1 Introduction](#)

[2 Motivation and Scope](#)

[3 Before/After Comparisons](#)

[4 Design Overview](#)

[5 Proposed Wording](#)

[6 References](#)

Introduction

Good error handling is important to all programming languages, not just C++. Since this is so fundamental to programming in general, small improvements can have tremendous gains. Likewise regressions, could set a programming language back. C++ is blessed with multiple ways of performing error handling. These range from throwing exceptions which is our standard, returning errors which we inherited from C (also our informal standard) and using globals/registers/thread local storage. The bifurcation in how errors are served up and consumed has caused some problems over the decades and have been the subject of many papers over the past few years. While many of these papers are geared toward healing the divide, they are primary focused on performance and error production instead of error consumption.

This paper however is evaluating a new feature for C++, pattern matching, and trying to ensure, in the context of error handling, it improves the language instead of indirectly widening the rift and in the process improve sometimes how we consume exceptions.

Motivation and Scope

It should be noted that is neither the motivation nor the scope of the current [pattern matching proposal](#) to provide any mechanisms for error handling. However, there are two parts of the pattern matching proposal that has a bearing on error handling.

The first one starts in section 4.4 Matching Variants. So if the return of a function is a variant of the expected value and the unexpected error code or object than the pattern matching proposal can be used to perform error handling.

```

variant<rete, error_code> e() noexcept;
variant<reto, error_object> o() noexcept;

variant<rete, error_code> temp1 = e();
inspect(temp1) {
    <rete> //...
    <error_code> //...
}
variant<reto, error_object> temp2 = o();
inspect(temp2) {
    <reto> //...
    <error_object> //...
}

```

The second is that while in most of the examples in the pattern matching proposal the *init-statement_{opt} condition* appears to be a variable it can also be a function call. The one example of this is found in 5.3.2.7 Extractor Pattern: Example "inspect (f()) {}". The previous example can be simplified further by removing the temporary variables.

```

variant<rete, error_code> e() noexcept;
variant<reto, error_object> o() noexcept;

inspect(e()) {
    <rete> //...
    <error_code> //...
}
inspect(o()) {
    <reto> //...
    <error_object> //...
}

```

The pattern matching proposal has excellent support for return based error handling. What does exception handling look like with the current pattern matching proposal?

```

reta a();// throws b, c, d

try {
    inspect(a()) {
        <reta> //...
    }
} catch(b) {
} catch(c) {
} catch(d) {
}

```

In other words, pattern matching has no support for exception based error handling. Worse, by indirectly favoring one type of error handling over the other it is widening the rift between return based and exception based error handling. Even worse still, it is favoring return based over the exception based; the non standard over the standard. Ideally, it would be a much more pleasant error handling experience if the exception handlers could be moved inside with the inspect handlers.

```

reta a();// throws b, c, d

inspect(a()) {
    <reta> //...
    <b> //...
    <c> //...
    <d> //...
}

```

The value of the conciseness of this request becomes more visually apparent when even more error handling is performed in close proximity. This is especially true when using 3rd party modules and their transitive

dependencies. All of which can vary in how errors are produced. See the more complicated example in the next section.

Before/After Comparisons

Given

```
reta a();// throws b, c, d
variant<rete, error_code> e() noexcept;
reti i();// throws f, g, h
variant<reto, error_object> o() noexcept;
retu u();// throws j, k, l
```

Before	After
<pre>try { inspect(a()) { <reta> //... } } catch(b) { } catch(c) { } catch(d) { } inspect(e()) { <retb> //... <error_code> //... } try { inspect(i()) { <reti> //... } } catch(f) { } catch(g) { } catch(h) { } inspect(o()) { <reto> //... <error_object> //... } try { inspect(u()) { <retu> //... } } catch(j) { } catch(k) { } catch(l) { }</pre>	<pre>inspect(a()) { <reta> //... //... <c> //... <d> //... } inspect(e()) { <rete> //... <error_code> //... } inspect(i()) { <reti> //... <f> //... <g> //... <h> //... } inspect(o()) { <reto> //... <error_object> //... } inspect(u()) { <retu> //... <j> //... <k> //... <l> //... }</pre>

Design Overview

What are the advantages to allowing inspect to catch exceptions?

1. inspect won't widen the error handling bifurcation
2. inspect won't widen the error handling bifurcation in favor of the less standard return based over the more standard exception based
3. programmers can use a single mechanism to handle return based and exception based error handling at the statement level

4. other than for performance reasons, the need to create dual error handling API's is diminished because the consumer can handle it the same way

NOTE: If it makes it any easier to implement the desired functionality, the *init-statement_{opt} condition* could be limited to a single function call, "inspect(a()) {}", instead of a compound call such as "inspect(a() + e() - i() * o() / u()) {}". This would be ok from a usability stand point because the programmer could always wrap the more complicated expression into a lambda.

Wouldn't we need some syntax to differentiate a type that could be both returned and thrown?

NO. Exceptions typically are derived from the exception base class and are thrown not returned. Returns tend to be error_code or something else which is not derived from exception. Accounting for a rarely used same type in both context scenarios would greatly diminish the majority case as a whole. The whole point of this syntax is the developer in statement based error handling doesn't need to know how the function/expression in question produced the error. Further this fringe outlier can easily be resolved by the implementer of said function. It is much easier for a library writer to refrain from a fringe case than to resort to providing dual interfaces.

**"How would we deal with the N+1 problem? Frequently new syntax is introduced as way to get rid of an older syntax, but frequently what happens is that we end up with the new syntax and the old syntax."
(source unknown)**

The current pattern matching proposal has already added new syntax which replaces in some circumstances if and switch. This proposal doesn't so much invent a new syntax as it removes a perceived limitation or restriction. While some may think this revision puts inspect in competition with try/catch, I assure it doesn't. It is actually complementary. Try/Catch is block based. Return based error handling as well as pattern matching's inspect is statement based. Block based error handling is ideal when dealing with blocks. Statement based error handling is ideal when dealing with statements. Which one the programmer uses depends on the situation at the moment of need! A situation may have try/catch as a catch all for a whole function block. However in that same function, a specific function call is required to handle the error at the moment of the call due to business requirements of the error handling occurring before some other statement. At that moment, programmers reach for a statement based tool even though there is some catch handler catching everything else. I believe there is a misconception that for dual API's programmer's reach for either exception or return based API calls. However if your code base uses both then you are more likely to use both.

Proposed Wording

If the *init-statement_{opt} condition* can throw an exception, then exceptions can be caught by inspect's *alternative-pattern*.

References

[P1371R2] Pattern Matching

<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2020/p1371r2.pdf>